Relazione

Base di Dati

Ho utilizzato una base di dati mongoDB per memorizzare i dati necessari al funzionamento dell'applicazione web. All'interno della base di dati sono presenti le seguenti collections:

- customers { _id, name, lastName, email, password, paymentMethod, preference, deleteDate }.
  Collection che memorizza i dati dei clienti. Durante la fase di accesso all'applicazione l'utente specifica il metodo di pagamento e una categoria dei piatti preferita.
- restaurateurs { _id, name, lastName, email, password, phone, pIva }.
  Collection che memorizza i dati dei ristoratori. Durante la fase di accesso all'applicazione l'utente specifica un numero di telefono e la partita Iva.
- restaurants { _id, owner, name, address, phone, description, openingTime, closingTime, deleteDate }.
  Collection che memorizza i dati dei ristoranti. Ciascun record collega tramite chiave esterna il ristorante al proprietario. L'indirizzo del ristorante é memorizzato come unico attributo, contiene: via, civico, città, cap.
- menus { _id, restaurantid, mealId, price, discount }
  Collection che memorizza i dati dei menu dei ristoranti. Ciascun record collega tramite chiavi esterne un piatto ad un ristorante, indicandone il prezzo di vendita. Il prezzo viene memorizzato all'interno di questa collection poiché ogni ristorante può vendere un piatto a prezzo diverso.
  I ristoratori sono in grado di aggiungere un sconto percentuale al prezzo del piatto per eventuali promozioni.
- meals { _id, name, category, area, instructions, prepTime, imgUrl, isSpecialty, ingredients, deleteDate }.
  Collection che memorizza i dati dei piatti. Ogni piatto ha un tempo di preparazione ed una categoria tra: primo, secondo e dessert. Tramite all'attributo booleano isSpecialty é possibile distinguere i piatti normali dalle specialità aggiunte dai singoli ristoranti.
- orders { _id, customerId, restaurantId, meals, totalPrice, prepTime, estimatedTime, status, orderTime, deliveryTime }.
  Collection che memorizza i dati degli ordini effettuati dai clienti. Ciascun record collega tramite chiavi esterne un cliente ad un ristorante, indicando il ristorante presso il quale il cliente ha richiesto un ordine. Vengono memorizzati i prezzi e le quantità di ogni piatto ordinato, insieme al prezzo totale. Inoltre si memorizza il tempo di preparazione dell'ordine

ed il tempo totale stimato, che tiene conto anche della coda di preparazione degli ordini del ristorante.

Le email sono considerate uniche tra clienti e ristoratori, quindi non possono esserci utenti che sono sia clienti sia ristoratori. L'unicità é garantita dai controlli effettuati nelle route di inserimento di un nuovo utente.

Eliminazione Logica

Per permettere una visualizzazione completa dello storico degli ordini, ho applicato un'eliminazione logica utilizzando l'attributo deleteDate sulle collection: restaurants, customers, meals.
In questo modo anche se un record in queste collection viene eliminato é comunque possibile recuperarne i dati.

Back-end

Per permettere la richiesta di più fetch simultaneamente, tramite la funzione `startServer()`, connetto l'applicazione alla base di dati all'avvio di node.
La connessione alla base di dati viene terminata quando viene terminato il processo di node.
Grazie a queste due funzioni in caso di fetch consecutive non si creano problemi di chiusura della connessione prematura.

Vediamo ora le diverse route utilizzate dall'applicazione web:

Autenticazione

- post - `/login` :
  Si occupa dell'accesso sia dei clienti sia dei ristoratori, sfruttando l'unicità dell'email.
  - Se l'utente non é stato trovato con le credenziali fornite restituisce : { error } (401)
  Altrimenti restituisce: { message, user } (200).

Clienti Routes

- post - `/customers` :
  Crea un nuovo cliente.
  - Se la validazione dei dati non passa restituisce: { error } (400).
  - Se l'email é già utilizzata tra clienti o ristoratori restituisce: { error } (409).
  Altrimenti inserisce un nuovo cliente e restituisce: { message, insertedId } (201).
- get - `/customers/:id` :
  Recupera i dettagli di un cliente dato il suo id.
  - Se l'id non é valido restituisce: { error } (400).
  - Se il cliente non é stato trovato restituisce: { error } (404).
  Altrimenti restituisce: { message, customer } (200).

- put - `/customers/:id` :
  Aggiorna il profilo del cliente dato il suo id.
  - Se la validazione dell'id ed almeno un campo fornito fallisce restituisce: { error } (400).
  - Se l'email data é già utilizzata tra clienti o ristoratori restituisce: { error } (409).
  - Se il cliente non é stato travato restituisce: { error } (404).
  Altrimenti aggiorna i dettagli del cliente e restituisce: { message } (200).

- delete - `/customers/:id` :
  Applica l'eliminazione logica di un cliente dato il suo id.
  - Se l'id non é valido restituisce: { error } (400).
  - Se il cliente non é stato trovato restituisce: { error } (404).
  Altrimenti elimina il cliente e restituisce: { message } (200).

Ristoratori Routes

- post - `/restaurateurs` :
  Crea un nuovo ristoratore.
  - Se l'email é già utilizzata tra clienti o ristoratori restituisce: { error } (409).
  - Se la validazione dei dati fallisce restituisce: { error } (400).
  Altrimenti inserisce un nuovo ristoratore e restituisce: { message, insertedId } (201).

- get - `/resturateurs/:id` :
  Recupera i dettagli di un ristoratore dato il suo id.
  - Se l'id non é valido restituisce: { error } (400).
  - Se il ristoratore non é stato trovato restituisce: { error } (404).
  Altrimenti restituisce: { message, restaurateur } (200).

- put - `restaurateurs/:id` :
  Aggiorna il profilo del ristoratore dato il suo id.
  - Se la validazione dell'id ed almeno un campo fornito fallisce restituisce: { error } (400).
  - Se l'email data é già utilizzata tra clienti o ristoratori restituisce: { error } (409).
  - Se il ristoratore non é stato travato restituisce: { error } (404).
  Altrimenti aggiorna i dettagli del ristoratore e restituisce: { message } (200).

- delete - `/restaurateurs/:id` :
  Elimina un ristoratore dato il suo id.
  - Se l'id non é valido restituisce: { error } (400).
  - Se il ristoratore non é stato trovato restituisce: { error } (404).
  Altrimenti elimina il ristoratore e restituisce: { message } (200).

Ristoranti Routes

- post - `/restaurants` :
  Crea un nuovo ristorante.

- Se la validazione dei dati fallisce restituisce: { error } (400).
- Se il ristoratore non é stato trovato restituisce: { error } (404).
- Se il ristoratore possiede già un ristorante restituisce: { error } (409).
Altrimenti inserisce un nuovo ristorante e restituisce: { message, insertedId } (201).

- get - `/restaurants/:id` :
Recupera i dettagli di un ristorante dato il suo id.
- Se l'id non é valido restituisce: { error } (400).
- Se il ristorante non é stato trovato restituisce: { error } (404).
Altrimenti restituisce: { message, restaurant } (200).

- get - `/restaurants/owner/:ownerId` :
Recupera i dettagli di un ristorante dato l'id del proprietario.
- Se l'id del proprietario non é valido restituisce: { error } (400).
- Se il ristorante non é stato trovato restituisce: { error } (404).
Altrimenti restituisce: { message, restaurant } (200).

- get - `/restaurants` :
Recupera i dettagli di tutti i ristoranti.
→ Se si specifica il parametro query search, i ristoranti saranno filtrati per nome o città.
→ Se si specifica il parametro query preference, ad ogni ristorante viene aggiunto un attributo che indica se sono presenti degli sconti sulla categoria indicata.
Se i ristoranti sono stati recuperati con successo restituisce: { message, restaurants } (200).

- get - `/restaurants/:id/stats` :
Recupera le statistiche di un ristorante dato il suo id.
- Se l'id non é valido restituisce: { error } (400).
- Se il ristorante non é stato trovato restituisce: { error } (404)
Altrimenti calcola e restituisce:
{ message, stats: { totalOrders, totalRevenue, pendingOrders, waitingTime } } (200).

- put - `/restaurants/:id` :
Aggiorna i dati di un ristorante dato il suo id.
L'indirizzo é trattato come un attributo atomico, quindi se si vuole aggiornare ogni sua parte deve essere fornita.
- Se la validazione dell'id ed almeno un campo fornito fallisce restituisce: { error } (400).
- Se il ristoratore non é stato travato restituisce: { error } (404).
Altrimenti aggiorna i dettagli del ristorante restituisce: { message } (200).

- delete - `/restaurants/:id` :
Applica l'eliminazione logica di un ristorante dato il suo id.
- Se l'id non é valido restituisce: { error } (400).

- Se il ristorante non é stato trovato restituisce: { error } (404).
Altrimenti elimina il ristorante e restituisce: { message } (200).

Menu Routes

- post - `/restaurants/:id/menu` :
Aggiunge un piatto al menu di un ristorante dato il suo id.
- Se la validazione dell'id e dei dati forniti fallisce restituisce: { error } (400).
- Se il ristorante ed il piatto non esistono restituisce: { error } (404).
- Se il piatto é già presente nel menu del ristorante restituisce: { error } (409).
Altrimenti inserisce il piatto nel menu del ristorante e restituisce:
{ message, insertedId } (201).

- get - `/restaurants/:id/menu` :
Recupera il menu di un ristorante dato il suo id.
Ad ogni record di menu aggiunge i dettagli del piatti.
- Se l'id non é valido restituisce: { error } (400).
- Se il ristorante non esiste restituisce: { error } (404).
Altrimenti restituisce: { message, menu } (200).

- put - `/restaurants/:restaurantId/menu/:mealId` :
Aggiorna i dati di un record del menu dato l'id del ristorante e del piatto.
- Se la validazione degli id ed almeno un campo fornito fallisce restituisce: { error } (400).
- Se il ristorante o il piatto non esiste restituisce: { error } (404).
Altrimenti aggiorna il menu e restituisce: { message } (200).

- delete - `/restaurants/:restaurantId/menu/:mealId` :
Elimina un record del menu dato l'id del ristorante e del piatto.
- Se l'id del ristorante o del piatto non é valido restituisce: { error } (400).
- Se il ristorante o il piatto non esiste restituisce: { error } (404).
Altrimenti elimina il record del menu e restituisce: { message } (200).

Piatti Routes

- post - `/meals` :
Crea una specialità di un ristorante.
- Se la validazione dei dati fallisce restituisce: { error } (400).
- Se il ristorante passato non esiste restituisce: { error } (404).
Altrimenti inserisce la specialità, inserisce il piatto nel menu del ristorante e restituisce:
{ message, insertedId } (201).

- get - `/meals` :
Recupera tutti i piatti che non sono specialità di un ristorante.
Se i piatti sono stati recuperati con successo restituisce: { message, meals } (200).

- get - `/meals/:id` :
  Recupera i dettagli di un piatto dato il suo id.
  - Se l'id del piatto non é valido restituisce: { error } (400).
  - Se il piatto non é stato trovato restituisce: { error } (404),
  Altrimenti restituisce: { message, meal } (200).

- put - `/meals/:id` :
  Aggiorna i dati di una specialità di un ristorante, dato l'id del piatto.
  Essendo l'aggiornamento di una specialità sempre legato anche al prezzo e sconto, questa route aggiorna anche i dati del menu.
  - Se la validazione degli id ed almeno un campo fornito fallisce restituisce: { error } (400).
  - Se il ristorante non esiste o se il piatto non é nel suo menu restituisce: { error } (404).
  - Se il piatto non esiste o se non é una specialità: { error } (404).
  Altrimenti aggiorna i dettagli del piatto, record del menu e restituisce: { message } (200).

- delete - `/meals/:id` :
  Elimina una specialità di un ristorante, dato l'id del piatto.
  - Se l'id non é valido restituisce: { error } (400).
  - Se il piatto non é una specialità del ristorante dato restituisce: { error } (400).
  - Se il piatto non esiste restituisce: { error } (404).
  Altrimenti elimina il piatto e restituisce: { message } (200).

Ordini Routes

- post - `/orders` :
  Crea un nuovo ordine per un cliente ad un ristorante.
  - Se la validazione degli id e dei dati forniti fallisce restituisce: { error } (400).
  - Se il ristorante o il cliente o uno dei piatti non esiste restituisce: { error } (404).
  - Se uno o più piatti non fanno parte del menu del ristorante restituisce: { error } (404).
  Altrimenti calcola il tempo di preparazione ed il prezzo totale, inserisce l'ordine e restituisce:
  { message, insertedId } (201).

- get - `/orders` :
  Recupera tutti gli ordini dei clienti filtrati secondo i seguenti parametri query.
  → id, indica l'id del cliente o del ristorante.
  → role, se ha valore "customer" vengono restituiti tutti gli ordini richiesti dal cliente, se ha valore "restaurateur" vengono restituiti tutti gli ordini ricevuti dal ristorante.
  Utilizza delle aggregazioni delle collections in modo da restituire tutti i dati necessari alla completa visualizzazione dell'ordine.
  - Se l'id o il ruolo fornito non é valido restituisce: { error } (400).
  Altrimenti restituisce: { message, orders } (200).

- put - `/orders/:id` :
  Aggiorna i dati di un ordine dato il suo id, in specifico lo stato dell'ordine.
  É possibile aggiornare unicamente lo stato di un ordine.
  - Se la validazione dell'id o dei dati fallisce restituisce: { error } (400).
  - Se l'ordine non é stato trovato restituisce: { error } (404).
  Altrimenti aggiorna lo stato dell'ordine e restituisce: { message } (200).

Per maggiori dettagli sui parametri e risposte delle api consultare lo swagger.

Funzionalità

Registrazione

> Pagina: index.html

Il form di registrazione viene utilizzato sia per registrazione dei clienti, sia per quella dei ristoratori.
Questo é composto da degli input tag in comune tra i due ruoli, ovvero { name, lastName, email, password }. In base al ruolo selezionato le select tag la funzione `changeInputFields()` aggiunge gli input tag per il cliente { paymentMethod, preference } , oppure quelli per il ristoratore { phone, plva }.

Una volta inviato il form viene chiamata la funzione `signIn()`, che in base al ruolo definisce l'url ed il body per chiamare la fetch. Gli url utilizzati da questa funzione con metodo **POST** sono:

- `'http://localhost:3005/customers'`
- `'http://localhost:3005/restaurateurs'`

Dopo la chiamata della fetch in base al ruolo viene creata la sessione con i dati dell'utente e si viene reindirizzati nella pagina iniziale dell'applicazione web.

La sessione degli utenti contiene { _id, role, username }, dove username é la concatenazione di nome e cognome. In caso l'utente sia un cliente viene aggiunta alla sessione anche la preferenza, invece in caso l'utente sia un ristoratore viene aggiunto alla sessione l'id del ristorante (questo viene inserito immediatamente dopo la sua creazione nella pagina `dashboardRest.html`).

Login

> Pagina: index.html

Sia i clienti, sia i ristoratori utilizzano lo stesso form di accesso, poiché la route `/login` con metodo **POST** é stata ideata per gestire l'accesso di entrambi i tipi di utente. Quindi il form presenta solamente due campi, ovvero {email, password}, ed al suo invio viene chiamata la funzione `logIn()`.

Questa funzione prende i valori dal form e chiama una fetch all'url `'http://localhost:3005/login'` , con metodo **POST**.
Successivamente alla chiamata della fetch in base al ruolo viene creata la sessione con i dati dell'utente e si viene reindirizzati nella pagina iniziale dell'applicazione web.

Informazioni dell'Utente

> Pagina: manageProfile.html

Le informazioni dell'utente sono caricate all'interno del form di modifica, come valore dell'attributo *placeholder* negli input tag, dalla funzione `loadUserProfile()`. Questa funzione viene chiamata al caricamento della pagina ed in base al ruolo dell'utente, preso dalla sessione, esegue una fetch ai seguenti due url, con metodo **GET**:

- `` `http://localhost:3005/customers/${userData._id}` ``
- `` `http://localhost:3005/restaurateurs/${userData._id}` ``

Dopo aver ricevuto i dati si popolano gli input tag in base al ruolo.

Il form presenta in questa pagina funziona allo stesso modo di quello di registrazione di un utente, é quindi utilizzabile sia da clienti, sia da ristoratori. Se dei valori vengono cambiati l'utente può inviare il form ed eseguire una fetch agli stessi url utilizzati precedentemente, ma con metodo **PUT**.

Ricerca dei Ristoranti

> Pagina: restaurants.html

La funzionalità di ricerca dei ristoranti prevede l'inserimento di un filtro nel recupero dei ristoranti.
É implementato un filtro secondo la città ed il nome del ristorante, questo può essere inserito all'interno di un form. All'invio del form viene chiamata la funzione `loadRestaurants()`.

Questa funzione, se lasciato vuoto l'input tag, carica nella pagina tutti i ristoranti, eseguendo una fetch all'url `` `http://localhost:3005/restaurants` `` `?preference=${userPreference}` , con metodo **GET**.
In caso il form venga compilato ed inviato viene aggiunto un ulteriore parametro query all'url, diventando
`` `http://localhost:3005/restaurants` `` `?searchTerm=${searchTerm}&preference=${userPreference}` .

Analizziamo nel dettaglio la route `/restaurants` con metodo **GET**, questa é composta da 2 parti.

- Creazione del filtro e esecuzione della query:
  Il filtro viene inizializzato con l'attributo deleteDate, in modo da non restituire i ristoranti che sono stati eliminati. Successivamente se é stato passato il parametro query *search*, viene aggiunto al filtro una query con un *$or*. In questo modo sono in grado di restituire i ristoranti che hanno il nome, oppure la città che inizia con il valore di *search*.

```
const { search, preference } = req.query;
let filter = { deleteDate: null };

if (search) {
    filter.$or = [
        { name: { $regex: new RegExp(`^${search}`, 'i') } },
        { address: { $regex: new RegExp(`, ${search}`, 'i') } }
    ];
}

const restaurants = await client
    .db('FastFood')
    .collection('restaurants')
    .find(filter)
    .toArray();
```

- Aggiunta dell'attributo `hasPreferredDiscount`:
  Dopo aver recuperato i ristoranti, per ciascun ristorante controllo i piatti inseriti nel menu e calcolo il valore dell'attributo `hasPreferredDiscount`, questo assume `true` se il ristorante ha almeno un piatto scontato della categoria preferita del cliente, `false` altrimenti.

```
if (preference && restaurants.length > 0) {
  for (let restaurant of restaurants) {
      const menuItems = await client
          .db('FastFood')
          .collection('menus')
          .find({ restaurantId: restaurant._id })
          .toArray();

      if (menuItems.length > 0) {
          const mealIds = menuItems.map(item => item.mealId);
          const meals = await client
              .db('FastFood')
              .collection('meals')
```

```
                .find({
                    _id: { $in: mealIds },
                    category: preference,
                    deleteDate: null
                })
                .toArray();

            restaurant.hasPreferredDiscount = menuItems.some(menuItem
=> {
                const meal = meals.find(m => m._id.toString() === men
uItem.mealId.toString());
                return meal && menuItem.discount && menuItem.discount
> 0;
            });
        } else {
            restaurant.hasPreferredDiscount = false;
        }
    }
}
```

Ricerca dei Piatti

> Pagina: addMeals.html

La funzionalità di ricerca dei piatti prevede l'inserimento di un filtro nel recupero dei piatti .
É implementato un filtro secondo la categoria ed il nome del piatto, questo può essere inserito all'interno di un form. All'invio del form viene chiamata la funzione `loadMeals()`.

Questa funzione, se lasciato vuoto l'input tag, carica nella pagina tutti i piatti (che non sono specialità), eseguendo una fetch all'url `'http://localhost:3005/meals'`, con metodo **GET**.
In caso il form venga compilato ed inviato viene aggiunto un ulteriore parametro query all'url, diventando: `` `http://localhost:3005/restaurants ?searchTerm=${searchTerm}` `` .

La route `/meals` con metodo **GET** si comporta allo stesso modo della prima parte del precedente punto.

Una volta recuperati i piatti, su ciascuno viene chiamata la funzione `createMealCard()` che si occupa della creazione delle card che verranno visualizzate sulla pagina.

Statistiche Ristorante

> Pagine: dashboardRest.html , restaurant.html

Le statistiche del ristorante vengono recuperate al caricamento di queste due pagine chiamando la funzione `loadRestaurantStats()`. La funzione esegue un fetch all'url `'http://localhost:3005/restaurants/${userData.restaurantId}/stats'` con metodo **GET**.

La route dopo aver recuperato tutti gli ordini ricevuti dal ristorante passato calcola le seguenti statistiche:

```
const orders = await client
.db('FastFood')
.collection('orders')
.find({ restaurantId: new ObjectId(id) })
.toArray();

const totalOrders = orders.length;
const totalRevenue = orders.reduce((total, order) => total + order.tot
alPrice, 0);
const pendingOrders = orders.filter(order => order.status === 'ordere
d' || order.status === 'in_progress');
const waitingTime = pendingOrders.reduce((total, order) => total + ord
er.prepTime, 0);
```

Carrello e Ordine

> Pagina: restaurant.html

Per questioni di leggibilità la logica del carrello si trova nel file *js/cart.js*.

Il carrello é una variabile salvata nella sessione del cliente che contiene { restaurantId, restaurantName, items, total, prepTime, restWaitingTime }.

Per aggiungere un prodotto al carrello si possono utilizzare i pulsanti presenti nelle card dei piatti (vedere restaurant.html). Una volta premuto il pulsante viene chiamata la funzione addToCart(), che aggiunge un elemento al carrello, ciascun elemento contiene { mealId, name, category, price, prepTime, quantity }.

Ogni volta che si aggiunge un elemento, oppure se ne modifica la quantità vengono ricalcolati il prezzo totale e il tempo di preparazione totale del carrello.
All'interno del carrello é possibile visualizzare il tempo di preparazione + il tempo di attesa.

Quando l'utente preme il pulsante di acquisto all'interno della sezione del carrello viene chiamata la funzione `processOrder()`. Questa funzione costruisce il body con i dati presenti nel carrello ed esegue una fetch all'url `'http://localhost:3005/orders'` con metodo **POST**.

Gestione Consegne

> Pagina: orders.html

Essendo che nella traccia sono presenti versioni contrastanti sullo stesso comportamento ho adottato il seguente metodo di gestione degli stati degli ordini:

- alla creazione dell'ordine questo ha lo stato di "ordered"
- quando il ristoratore indica che é in preparazione l'ordine assume lo stato di "in_progress"
- quando il ristoratore indica che la preparazione é completata l'ordine assume lo stato di "delivered".

Il ristoratore può cambiare lo stato di un ordine interagendo con la mappa dello stato dell'ordine (vedere orders.html). Premendo un pulsante viene chiamata la funzione `updateOrderStatus()`, che si interfaccia con la API, eseguendo una fetch all'url `'http://localhost:3005/orders'` con metodo **PUT**.

Quando il ristoratore cambia lo stato di un ordine in "delivered" , si presuppone che la preparazione dei piatti sia terminata e l'ordine venga immediatamente consegnato. Oltre a modificare lo stato viene anche aggiunta una data di consegna.

Storico Ordini

> Pagina: orders.html

Sia i ristoratori sia i clienti possono visualizzare lo storico degli ordini ricevuti / richiesti.

La funzione `loadOrders()` carica gli ordini che coinvolgono l'utente, in base al ruolo e l'id, infatti esegue una fetch all'url `'http://localhost:3005/orders?role=${role}&id=${id}'` con metodo **GET**.

Analizziamo nel dettaglio la route `/orders` con metodo **GET**, questa utilizza delle aggregazioni, tramite *lookup*, per restituire i dati di un ordine, assieme a quelli dell'utente/ristorante interessato e i piatti acquistati in esso.

Prima di tutto si aggiunge un filtro in base al ruolo dell'utente, ad esempio un ristoratore é in grado di visualizzare l'utente che ha richiesto l'ordine.

```
let filter = {};
  let lookupUser;
  if (role === 'restaurateur') {
      filter.restaurantId = new ObjectId(id);
      lookupUser = {
          $lookup: {
              from: 'customers',
              localField: 'customerId',
              foreignField: '_id',
              as: 'customer'
```

```
            }
        };
    } else if (role === 'customer') {
        filter.customerId = new ObjectId(id);
        lookupUser = {
            $lookup: {
                from: 'restaurants',
                localField: 'restaurantId',
                foreignField: '_id',
                as: 'restaurant'
            }
        }
    } else {
        return res.status(400).json({ error: "Ruolo non valido" });
    }
```

Successivamente viene eseguita la query aggiungendo una *lookup* per i piatti acquistati.

```
const orders = await client
    .db('FastFood')
    .collection('orders').aggregate([
        { $match: filter },

        lookupUser,

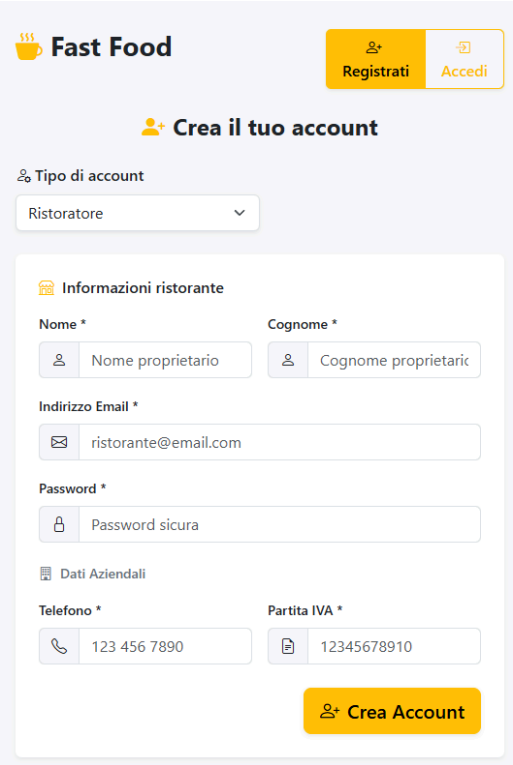
        {
            $lookup: {
                from: 'meals',
                localField: 'meals.mealId',
                foreignField: '_id',
                as: 'mealsDetails'
            }
        },
    ]).toArray();
```

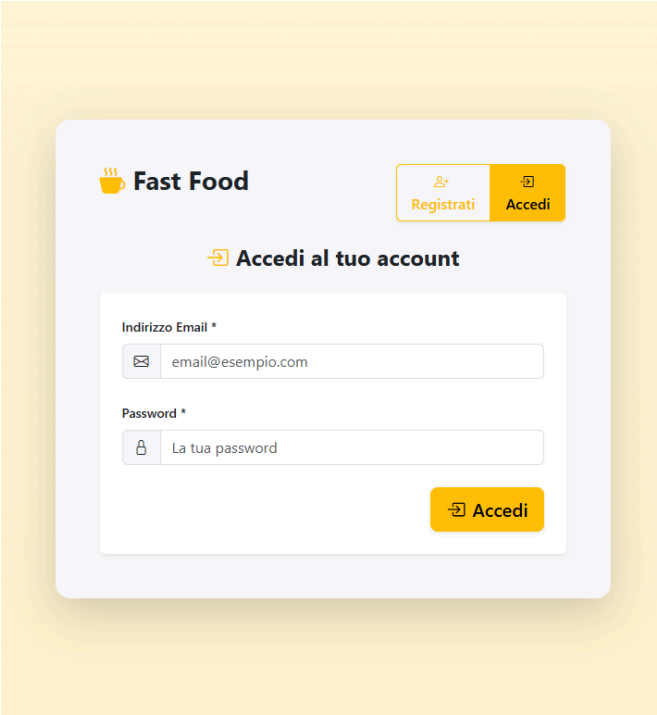
Dopo aver recuperato correttamente gli ordini, per ciascuno di essi viene creata una card con la funzione `createOrderCard()` e la mappa dello stato con la funzione `setupStatusRoadmap()`.

Prove di Funzionamento

index.html

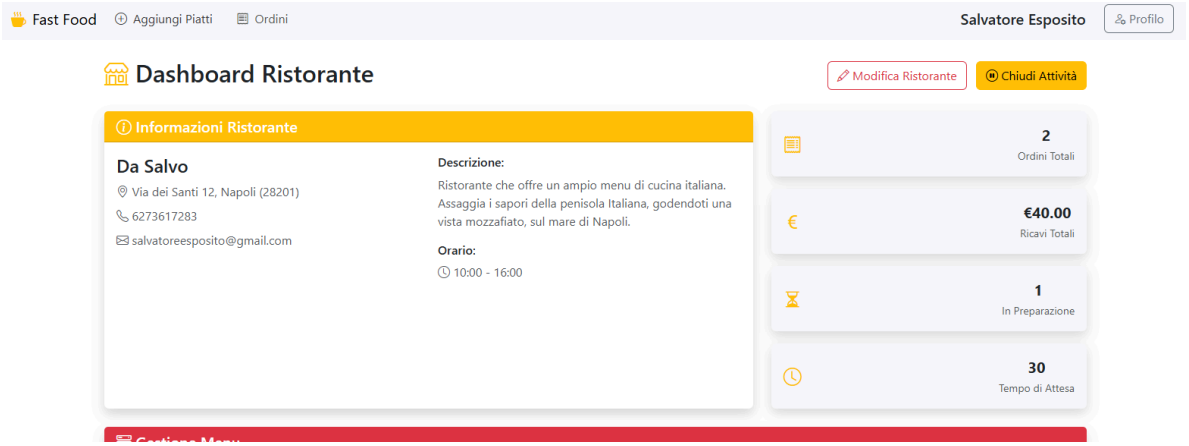
La prima pagina dell'applicazione web, alla quale si viene reindirizzati se non si ha ancora effettuato il *log-in*. Presenta un form di *sign-in* alternabile tramite un select tra cliente e ristoratore.
É possibile passare al form di *log-in* grazie ai pulsanti presenti in alto a destra.

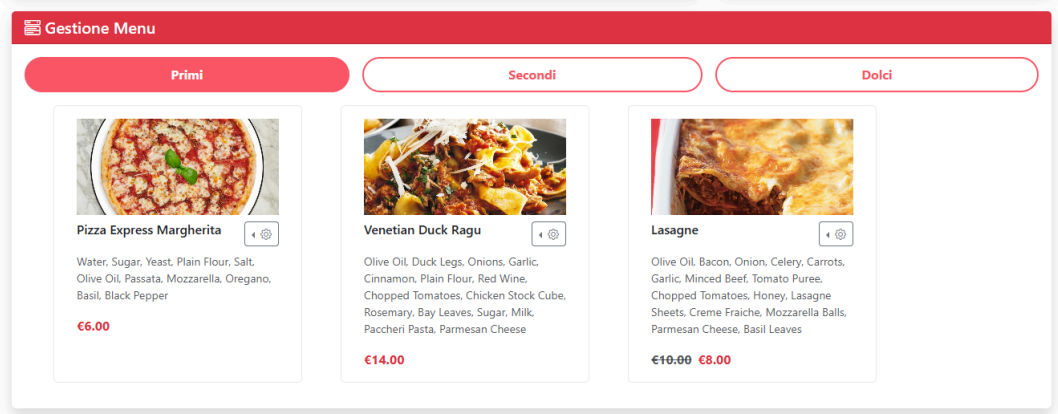




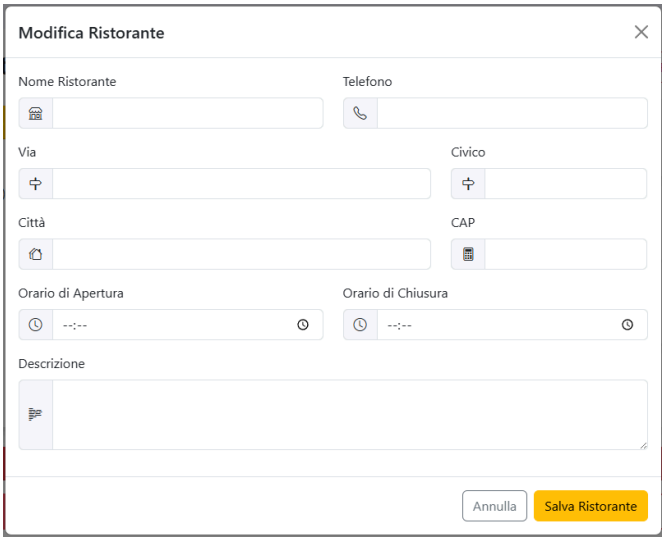
dashboardRest.html

Questa é la pagina alla quale si viene portati dopo aver effettuato il log-in come ristoratore. In questa pagina il ristoratore può visualizzare le informazioni, le statistiche ed il menu del proprio ristorante.



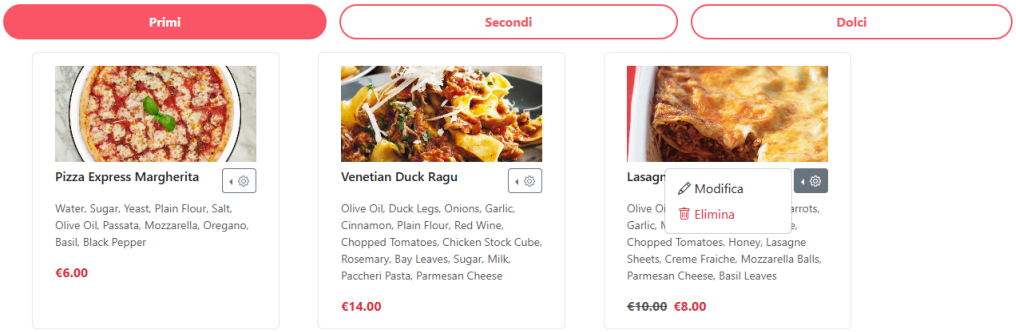


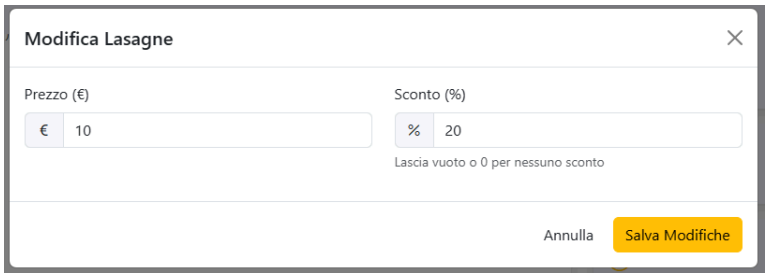
Il ristoratore può decidere di modificare oppure eliminare il proprio ristorante utilizzando i pulsanti in alto a destra, questi aprono i seguenti modali:



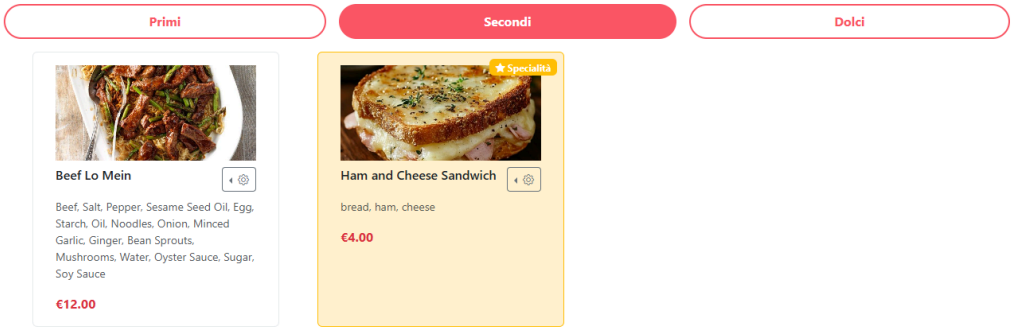


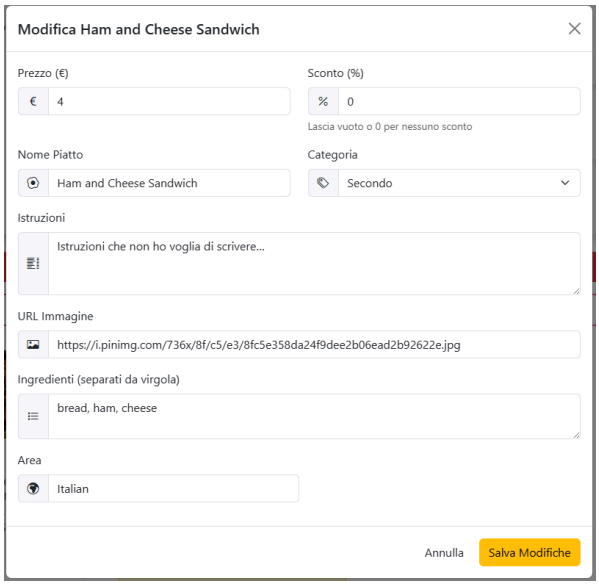
É possibile modificare il prezzo o aggiungere uno sconto ad ogni piatto, inoltre é possibile eliminare un piatto dal menu del ristorante.





Le specialità del ristorante si notano, poichè hanno uno sfondo ed un badge giallo.
In caso il piatto che si vuole modificare sia una specialità, allora il ristoratore sarà in grado di modificarne tutti i dati. Inoltre qualora il piatto da eliminare sia una specialità, allora questo non sarà solo rimosso dal menu ma anche eliminato dalla collection meals.

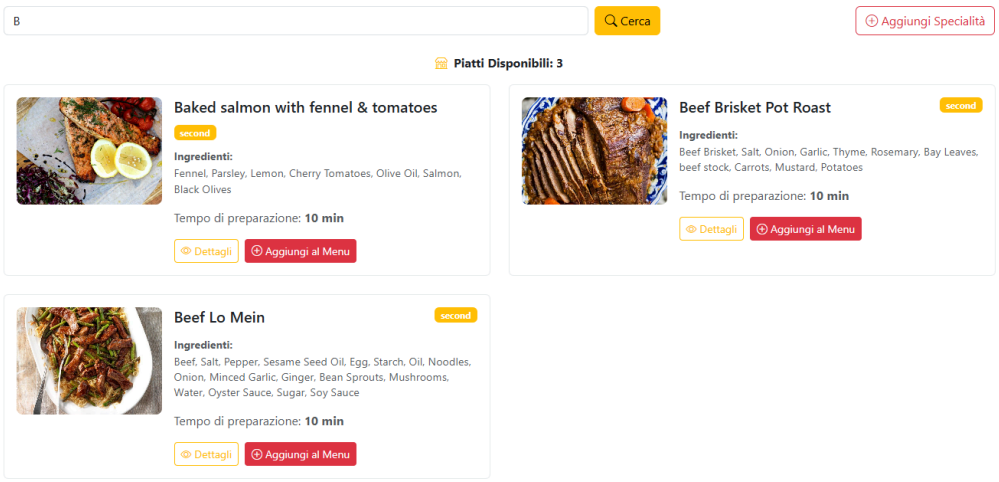


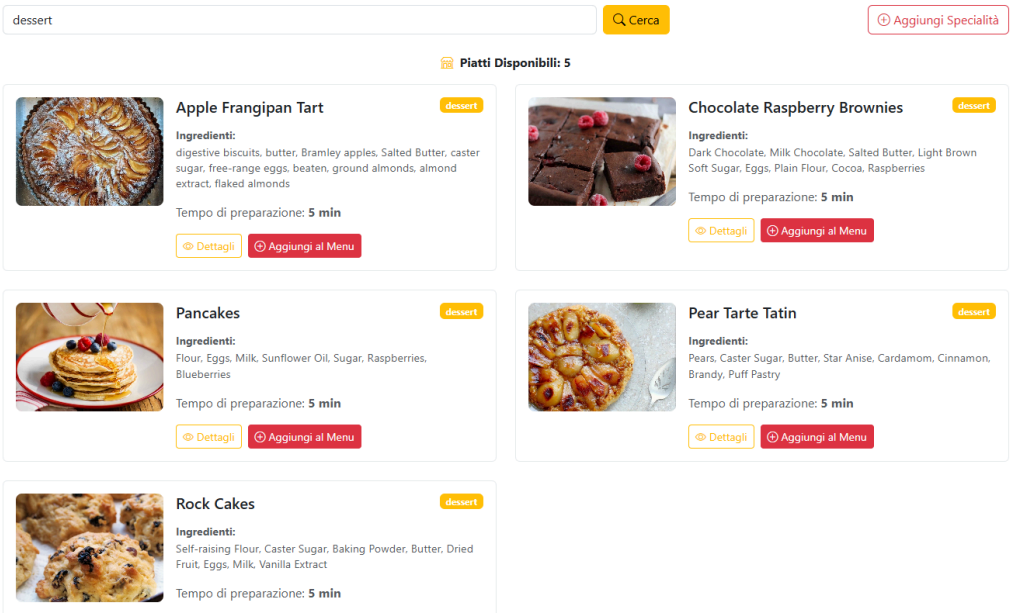




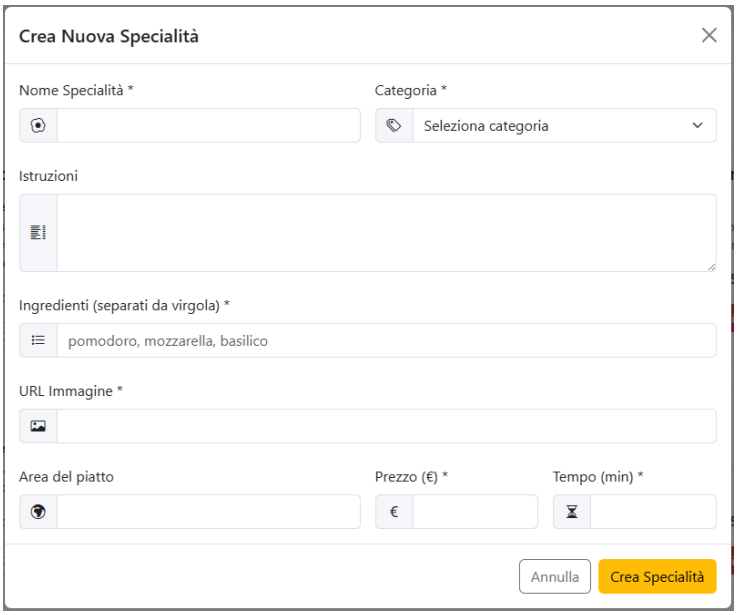
addMeals.html

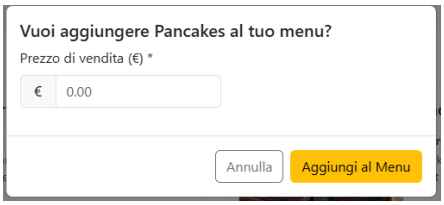
Tramite i link della navbar é possibile raggiungere questa pagina, dove il ristoratore può aggiungere dei piatti al proprio menu. Normalmente vengono mostrati tutti i piatti che non sono specialità, ma é possibile filtrare la ricerca per nome oppure categoria:





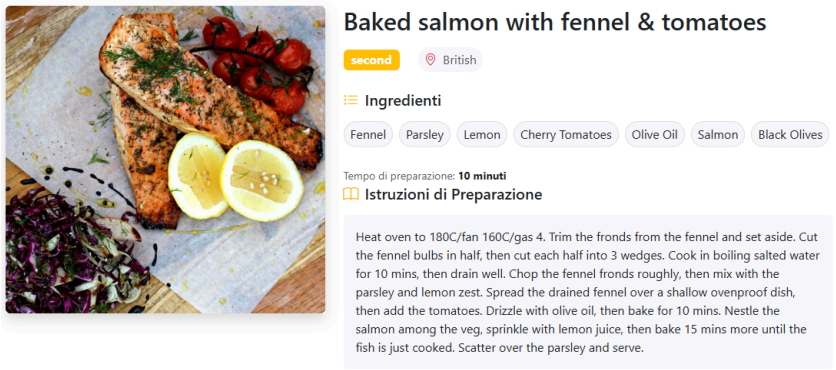
Il pulsante aggiungi al menu presente su ciascun piatto apre un modale per l'inserimento del record del menu. Invece se si vuole inserire una specialità si può usare il pulsante di fianco alla barra di ricerca.





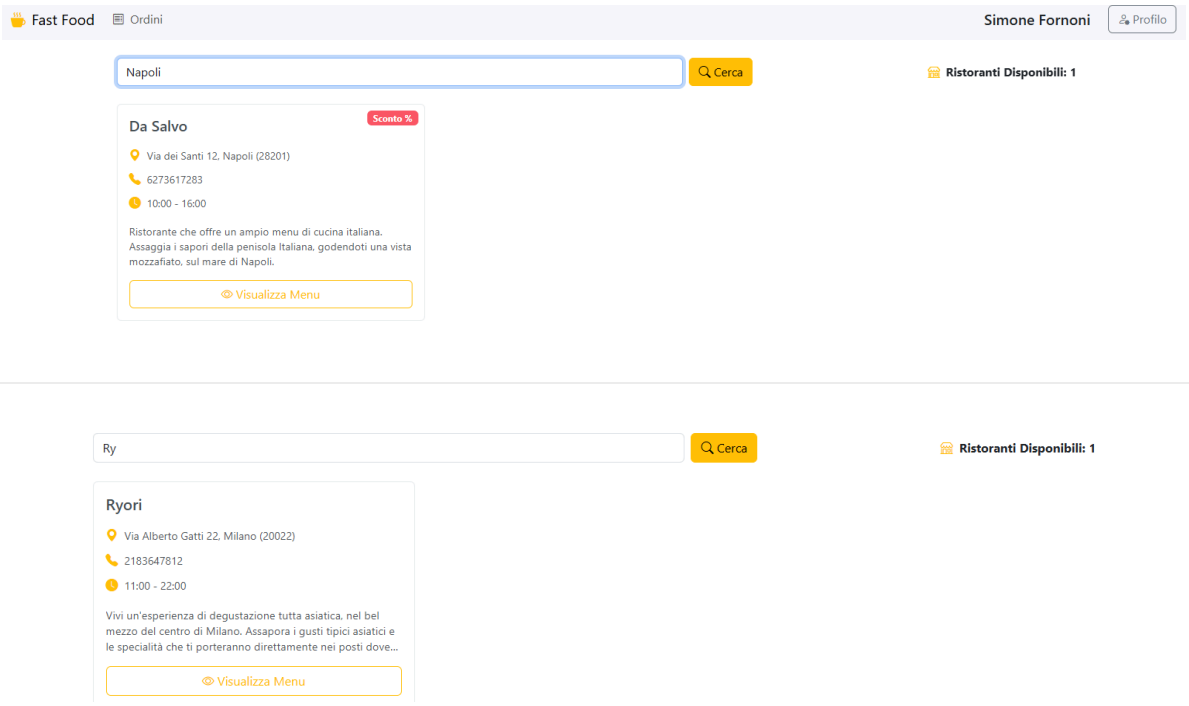
meal.html

Premendo il pulsante dettagli presente su ciascun piatto si raggiunge questa pagina, che mostra tutti i dettagli di un piatto.



restaurants.html

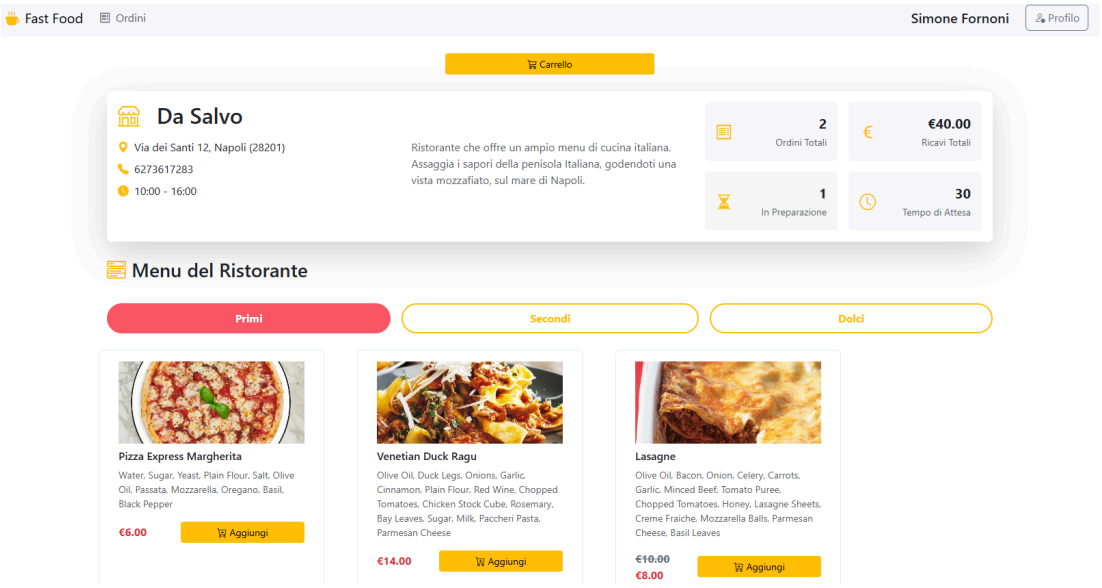
Questa é la pagina alla quale si viene portati dopo aver effettuato il log-in come cliente. In questa pagina il cliente può scegliere il ristorante nel quale vuole ordinare dei piatti. Normalmente sono mostrati tutti i ristoranti ma possono essere filtrati tramite la barra di ricerca secondo la città o il nome.
I ristoranti che hanno sconti su piatti della categoria segnata come preferenza dal cliente vengono indicati con il badge rosso.



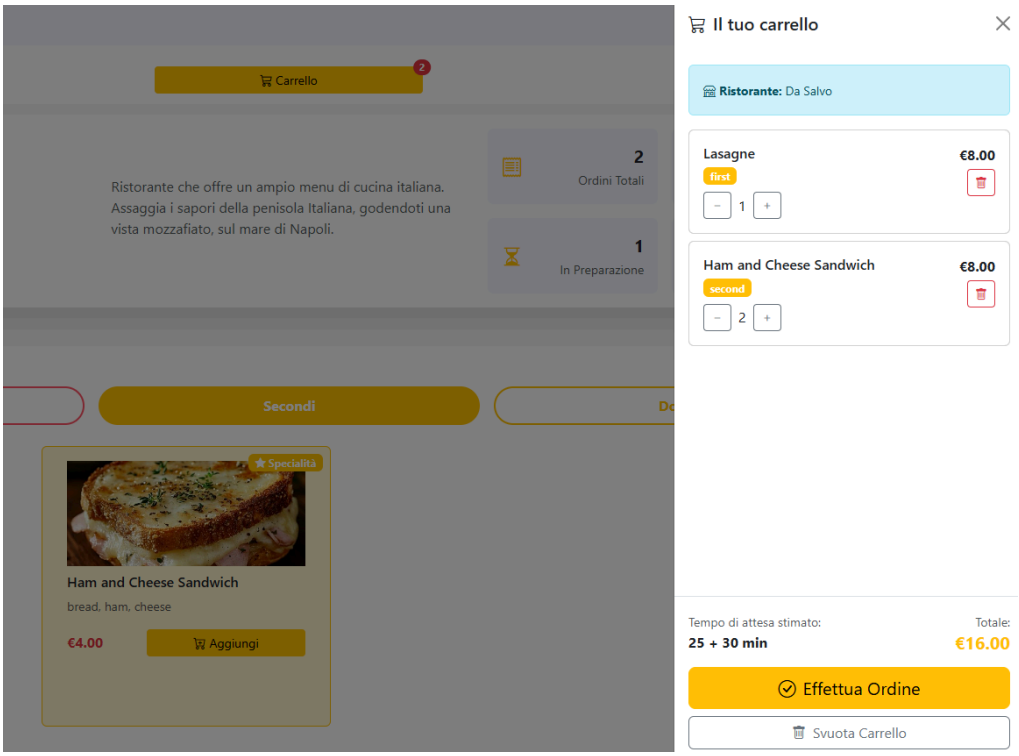
restaurant.html

Premendo il pulsante visualizza menu si accede alla pagina dedicata al ristorante, che permette al cliente di vedere i piatti nel menu del ristorante ed aggiungerli al carrello.

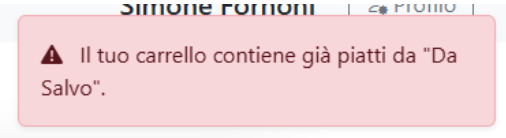
La categoria di piatti segnata come preferita dall'utente é colorata in rosso ed é impostata come attiva al caricamento della pagina.



Il carrello é accessibile sia dalla pagina restaurants.html sia dalla pagina restaurant.html. Per aggiungere i piatti al carrello di preme il pulsante aggiungi. Il carrello permette di visualizzare il ristorante presso il quale si sta componendo l'ordine, i piatti aggiunti, con la possibilità di modificarne la quantità. Inoltre é presente il prezzo totale ed il tempo di attesa, nel formato:
tempo di preparazione + tempo di attesa del ristorante.



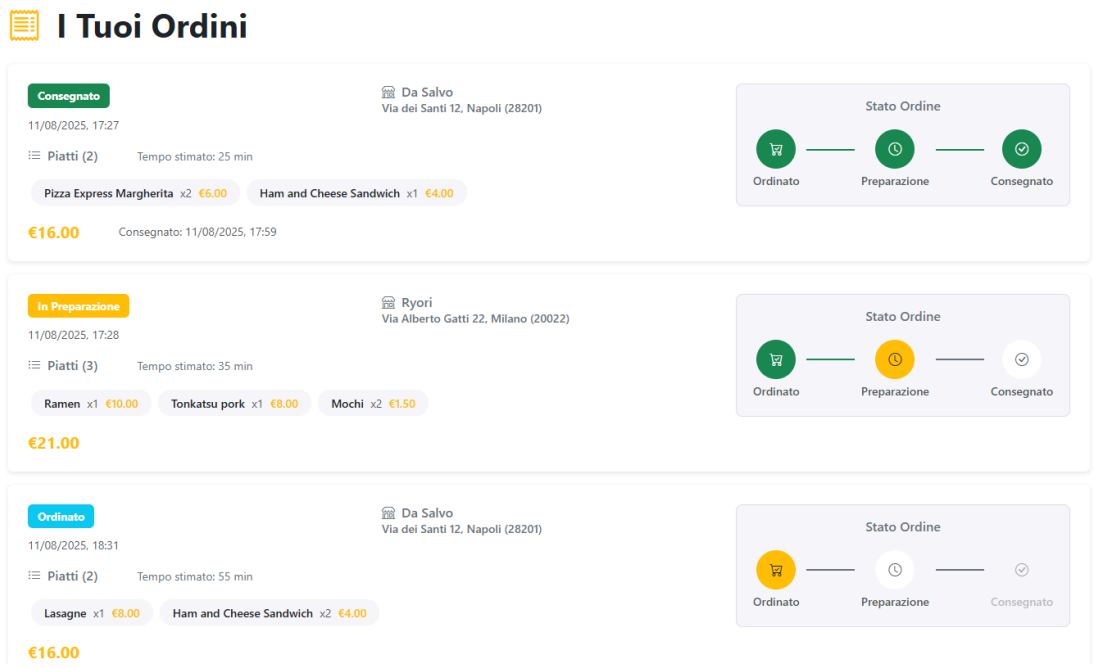
Se si prova ad aggiungere un piatto da un altro ristorante si viene avvertiti grazie ad un alert:



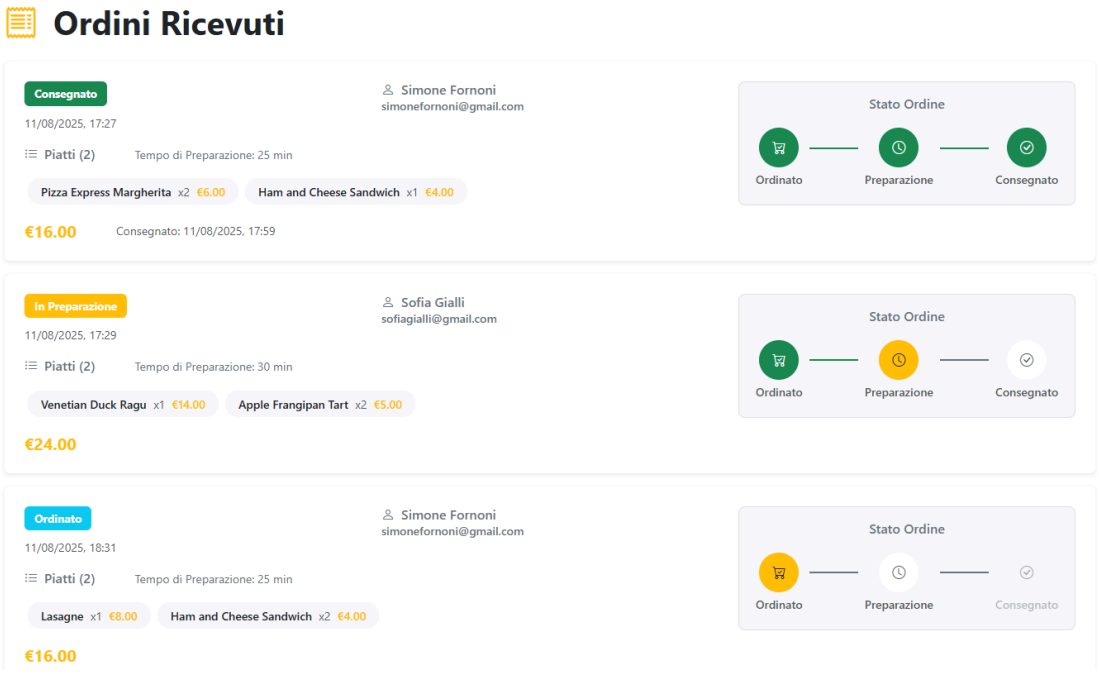
orders.html

Questa pagina é accessibile tramite il link nella navbar da entrambi i tipi di utente.

Dalla parte del cliente possiamo vedere tutte le specifiche degli ordini richiesti, assieme allo stato, che non può essere modificato.



Dalla parte del ristoratore possiamo vedere tutte le specifiche degli ordini ricevuti, assieme allo stato, che può essere modificato.

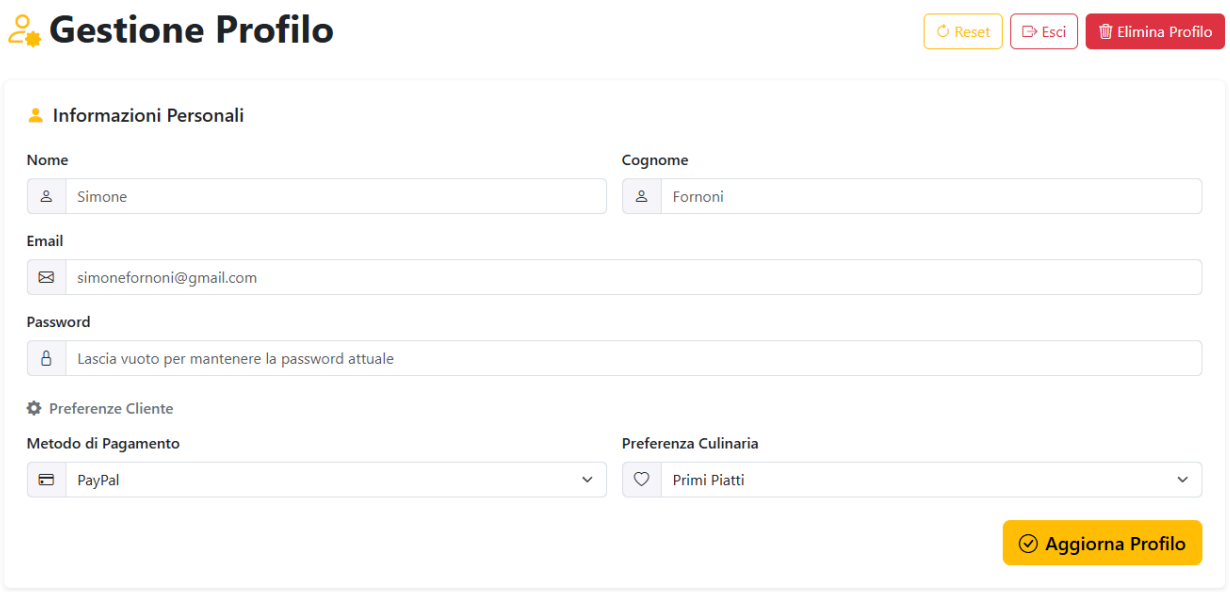


manageProfile.html

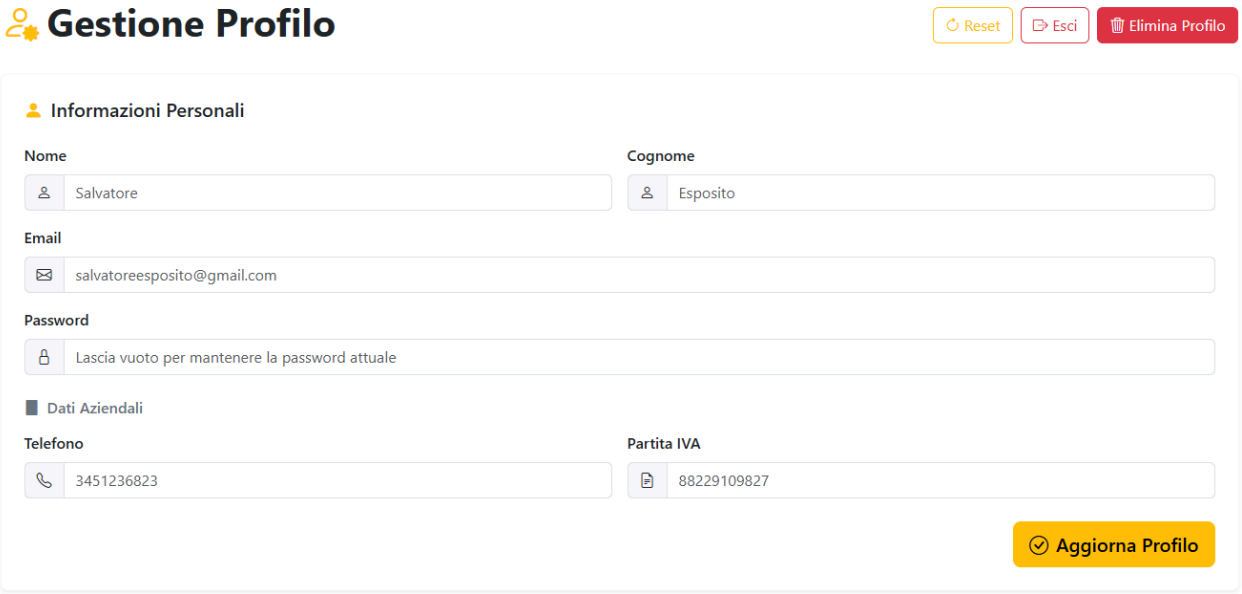
Questa é un’altra pagina accessibile da entrambi i tipi di utente.

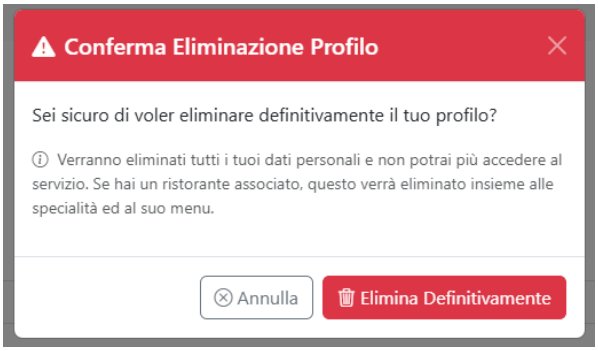
L'utente può visualizzare e modificare le proprie informazioni tramite il form, oppure effettuare il logout, oppure eliminare il profilo.

Cliente:



Ristoratore




Conferma Eliminazione Profilo
Sei sicuro di voler eliminare definitivamente il tuo profilo?
Verranno eliminati tutti i tuoi dati personali e non potrai più accedere al servizio. Se hai un ristorante associato, questo verrà eliminato insieme alle specialità ed al suo menu.
Annulla
Elimina Definitivamente